

FLASH-MAS – A Flexible and Lightweight Agent Shell

April 17, 2026

Contents

1	Concepts and terminology	2
2	Functionality	2
2.1	Hello World – A simple Java agent	3
2.2	Configuring entities	4
2.3	Transmission of messages between entities	7
2.3.1	<i>Waves</i>	7
2.3.2	Using communication pylons directly	8
2.3.3	Using communication pylons via messaging shards	10
2.3.4	Starting the deployment	11
3	Maven	11
3.1	Build	13
3.2	Tests	13
3.3	Build and tests	13
3.4	Configuration of source directories	13
3.5	Upgrade of dependencies	14
3.6	Generating a jar with all dependencies	14

1 Concepts and terminology

- agent
- entity
- deployment
- message
- context

2 Functionality

2.1 Hello World – A simple Java agent

Writing a new agent in FLASH-MAS is as simple as extending the `BaseAgent` class and overriding the `start()` method. Extending `BaseAgent` is not mandatory, but implementing the `Agent` interface would require implementing more methods, as described in section [TODO: ref section entities / EntityBase].

```
import net.xqhs.flash.core.agent.BaseAgent;

public class HelloWorldAgent extends BaseAgent {
    public boolean start() {
        if (!super.start()) return false;
        li("Hello World");
        stop();
        return true;
    }
}
```

Figure 1: Hello World agent.

The code for the Hello World agent is shown in Figure 1. The implementation uses the `li` method for logging, a mechanism which is detailed in section [TODO: ref Logging], but it could have used `System.out.println()` just the same.

The Hello World example agent is implemented in package `src-examples/example`.

For deploying the Hello World agent, one could use one of two possibilities of configuring a deployment (more details on configuring deployments, or deploying without a configuration, in Section [TODO: ref deployment configuration]): via command line arguments or via an XML deployment file. These two manners of creating a deployment are illustrated in the classes `BootDeployment` and `BootCLI` in the same package as the Hello World agent.

[TODO: document usage using the FLASH-MAS jar, not the repository itself] [TODO: mention generating of lib-all if using the sources]

To start the deployment directly from the system terminal, in the root of the repository, one can run, for the two variants, respectively¹:

```
java -cp "bin:lib-all/*" quick.Boot -agent HelloWorldAgent classpath
:example.helloWorld.HelloWorldAgent
```

```
java -cp "bin:lib-all/*" quick.Boot src-examples/example/helloWorld/
deployment.xml
```

¹In Windows, semi-colon must be used instead of a colon in the value for `-cp`.

```

import net.xqhs.flash.core.agent.BaseAgent;

public class ConfigurableHelloWorldAgent extends BaseAgent {

    public boolean start() {
        if (!super.start()) return false;

        Timer t = new Timer();
        t.schedule(new TimerTask() {
            @Override
            public void run() {
                stop();
                t.cancel();
            }
        }, stopAfter);

        return true;
    }
}

```

Figure 2: Configurable Hello World agent.

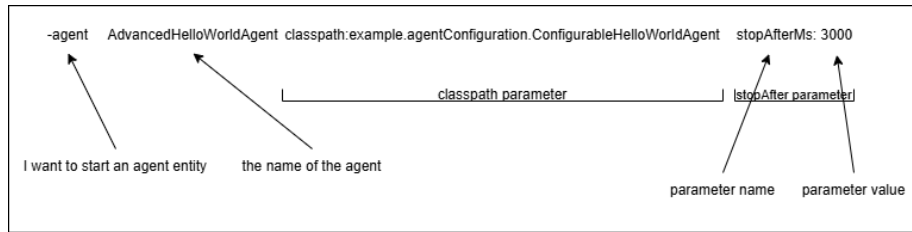


Figure 3: Configuration of parameters of an entity via command line arguments.

2.2 Configuring entities

The Agent Configuration example is implemented in package `src-examples/example`. The difference between the Agent Configuration example and the previous one is the implementation of the overridden `start()` method.

The code for the Configurable Hello World agent is shown in Figure 2. The implementation uses the `stopAfter` parameter and `Timer` class to schedule when the agent should stop. We will now present in detail how parameters for an entity can be configured in the deployment and how they can be extracted by the entity from that configuration.

The parameters of an entity can be *sent* via the following two variants:

- in a deployment configured via the command line. Figure 3 presents the configuration needed to start the deployment via command line arguments: the name of the agent, classpath and `stopAfter` parameters.
- in a deployment configured in an XML deployment file. Figure 4 presents the parameters needed to start a deployment via XML deployment file, being the same as above.

The Configurable Hello World agent is implemented in two variants, the difference being how the parameters are extracted from the configuration, presented below.

```
<deployment>
  <agent classpath="example.agentConfiguration.
    ConfigurableHelloWorldAgent" name="HelloWorldAgent">
    <parameter name="stopAfterMs" value="3000"/>
  </agent>
</deployment>
```

Figure 4: Configuration of parameters of an entity via XML deployment file.

There are also two variants through which the parameters received in the configuration can be extracted by an entity:

- extending the `EntityCore` class and using the `getConfiguration()` method. Figure 5 presents the code in this scenario. The implementation uses the `getConfiguration()` method for extracting `stopAfter` parameter.
- overriding the `configure()` method. Figure 6 presents the code in this scenario. The implementation overrides the `configure()` method and extracts the `stopAfter` parameter. The `configure()` method of an entity is called immediately after its construction.

```
public class ConfigurableHelloWorldAgent extends BaseAgent {
    public boolean start() {
        if(!super.start()) return false;

        long stopAfter = getConfiguration().get(
            STOP_AFTER.PARAMETER_NAME).orElse(DEFAULT.STOP_AFTER);

        Timer t = new Timer();
        t.schedule(new TimerTask() {...}, stopAfter);
        return true;
    }
}
```

Figure 5: Configurable Hello World Agent extending `EntityCore` class and using `getConfiguration()` method.

It can be observed that the `configure()` method receives, as parameter, the configuration, as a `MultiTreeMap` object. The `getConfiguration()` method returns an object of the same type.

The `MultiTreeMap` class acts as a tree of key-value pairs that allows multiple values for the same key. Keys are also called names. The class builds on the same principles as the `MultiValueMap` but acts rather like a hierarchical `MultiValueMap`. It has two types of names. Some names are "simple", in that they act exactly like names with `String` values in the `MultiValueMap`. There is a name and one or more `String` values associated with the name. Other names are "hierarchical"

```

public class ConfigurableHelloWorldAgent2 extends BaseAgent {

    @Override
    public boolean configure(MultiTreeMap configuration) {
        if(configuration.isSet(STOP_AFTER_PARAMETER_NAME))
            stopAfter = configuration.getFirstValue(
                STOP_AFTER_PARAMETER_NAME);
        else
            stopAfter = DEFAULT.STOP_AFTER;
        return true;
    }
}

```

Figure 6: Configurable Hello World Agent implementing `configure()` method.

(or tree-keys). Each hierarchical name has one or more associated **MultiTreeMap** values. The class extends the **MultiValueMap** such that only the two types of values are admitted: String and the **MultiTreeMap**. One name can be associated either with String values, or with tree values, but not with a combination thereof (a name must either be simple or hierarchical). There is no intersection allowed between the set of names for simple values and the set of types for tree values. For clearer implementation and easier debugging, there are separate methods for the names which are supposed to only hold one value (call them singleton names), both for simple and for tree names. It is possible to change the singleton status of names by calling `makeSingleton`.

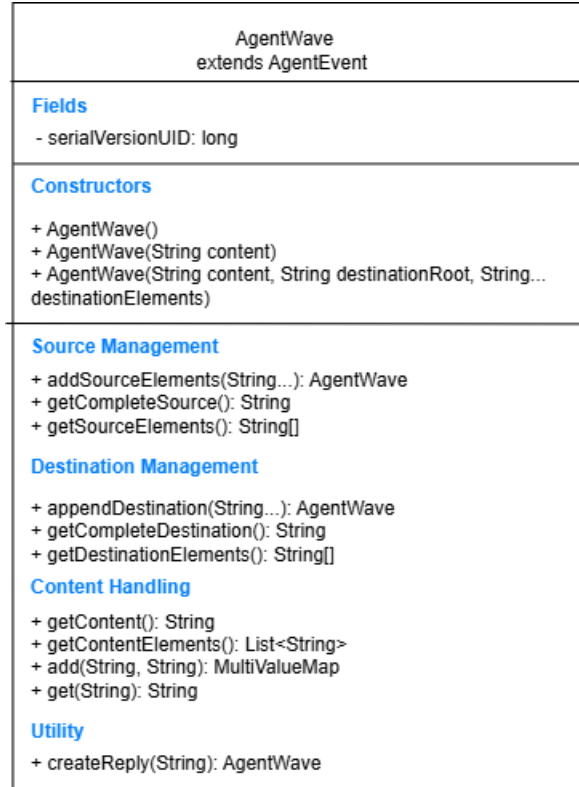


Figure 7: **AgentWave** Class Diagram in which the most important methods were included.

2.3 Transmission of messages between entities

Several examples of message transmission are implemented in subpackages of `src-examples/example`. In all of them, the scenario is of two or more agents which send ping-pong messages between them. Ping agent sends a ping every one second to the pong agent and the pong agent replies with a "pong" message to the ping agent. This process ends when the predefined limit has been reached.

2.3.1 Waves

In FLASH-MAS, an interaction between entities is performed via a *wave*, implemented in the class **AgentWave**.

Figure 7 presents the class diagram of **AgentWave** in which the most important methods were included.

A wave has a *source*, a *destination*, and *content*. Waves are transmitted between entities either directly (via method calling) or by means of a communication infrastructure, which can transmit waves inside the same node or between

different nodes.

The source and destination are *endpoints*. An endpoint is represented as a path made of multiple *elements*, separated by the character /. For example, an endpoint could look like `node1/agentA`, including the node name and the agent names, but the elements included in an endpoint before an agent (or, in general, entity) name depend strongly on the communication infrastructure used.

A simple `AgentWave` can be created using its default constructor, or with content and the elements in the destination, e.g.:

```
AgentWave wave = new AgentWave("hello", "node1", "agentA");
```

The source and destination of a wave can be configured either as full addresses (called *complete* source/destination) or incrementally, using individual elements (called source/destination *elements*). There are methods in the `AgentWave` class for

- element-wise source management: `addSourceElements()`, `getCompleteSource()`, `getSourceElements()`
- element-wise destination management `appendDestination()`, `getCompleteDestination()`, `getDestinationElements()`
- computing or resetting the complete source / destination endpoints: `createReply()`, `makePath()`, `pathToElements()`

To add content to a wave, one can use the `add()` method or provide it through the constructor. Since `AgentWave` extends `MultiValueMap`, it supports multiple key-value pairs for content. By default, content is associated with the “content” key², but other content keys can be added via `add()`. The method `getContentElements()` returns all content-related keys, except those used internally for routing.

A *message* is a wave sent between two agents, via a communication infrastructure. A *Pylon* is an *embodiment* of a communication infrastructure on a node. Interaction with a communication infrastructure is done directly (via proxy to the pylon) or via a *messaging shard*, which may be custom for the pylon implementation.

2.3.2 Using communication pylons directly

We take an example in which two instances of the same agent class, `AgentPingPongPlain`, communicate back and forth by sending and receiving waves through a pylon, which is accessed by entities via a proxy (the `WaveMessagingPylonProxy` class). One instance is configured to act as the “ping” initiator (it has a `sendTo` parameter pointing to the other agent’s name), while the other implicitly becomes the “pong” responder (it has no `sendTo` parameter).

²Stored as a string in the `AgentWave.CONTENT` constant

Initialization The agent is placed (using information from the deployment configuration) in the context of the pylon. The pylon is visible by the agent via a proxy. When added in the context of the pylon, the agent *registers* to the pylon – what it means to be registered – the communication infrastructure knows the node where the agent is located and the name of the agent, so it can receive messages; agent must provide a proxy to the pylon, as a `WaveReceiver` instance.

Figure 8 presents the essential code for the initialization stage. The complete implementation can be seen in the `AgentPingPongPlain` class.

```
@Override
public boolean addContext(EntityProxy<Pylon> context) {
    pylonProxy = (WaveMessagingPylonProxy) context;
    return pylonProxy.register(getName(), new WaveReceiver() {
        @Override
        public void receive(AgentWave wave) {
            processEvent(wave);
        }
    })
}
```

Figure 8: The essential code for the initialization stage

Sending messages The `AgentPingPongPlain` agent sends waves periodically using a timer. When the agent starts and it has a list of destinations, it schedules a task that sends a message to each destination every second. Each message contains a ping count and optionally the keyword `last` when the limit has been reached. The messages are constructed using the `AgentWave` class and sent via the `WaveMessagingPylonProxy`.

Figure 9 presents the essential code for the sending messages stage. The complete implementation can be seen in the `AgentPingPongPlain` class.

```
protected void sendPing() {
    AgentWave wave = new AgentWave("ping-no-" + tick, otherAgent, "pong");
    pylonProxy.send(wave);
}
```

Figure 9: The essential code for the sending messages stage

Receiving messages When an `AgentPingPongPlain` agent is registered to the pylon, it provides a receiver implementation that is triggered on incoming waves. Upon receiving a wave, the agent inspects its content and, if it is a pong agent – it was not initialized with a list of agents to ping – it constructs and sends a reply. Replies use the `createReply()` method, which sets the destination to the original source of the message. If the received message contains the keyword `last`, the agent stops after replying.

Figure 10 presents the essential code for the receiving messages stage. The complete implementation can be seen in the `AgentPingPongPlain` class.

```
protected boolean processEvent(AgentWave wave) {
    String replyContent = wave.getContent() + "-reply";
    return send(wave.createReply(replyContent));
}
```

Figure 10: The essential code for the receiving messages stage

2.3.3 Using communication pylons via messaging shards

We take the same ping-pong example, but here the agent delegates all messaging to a `MessagingShard` rather than using the `WaveMessagingPylonProxy` directly.

Instantiating shard The shard is instantiated in the `register()` method using the `instantiateRecommendedShard()` method from the `AgentShardCore` class. After instantiation, the shard is bound to the context, which gives it access to the messaging system through the proxy.

Figure 11 presents the essential code for the instantiating shard stage. The complete implementation can be seen in the `AgentPingPong` class.

```
msgShard = AgentShardCore.instantiateRecommendedShard(MESSAGING,
    (PylonProxy) context, null, new BaseAgentProxy() {
        @Override
        public boolean postAgentEvent(AgentEvent event) {
            return processEvent(event);
        }
    });
msgShard.addGeneralContext(context);
```

Figure 11: The essential code for the instantiating shard stage

Signaling start event When the agent starts, it tells the shard by sending it an `AGENT_START` event.

Figure 12 presents the essential code for the signaling start event stage. The complete implementation can be seen in the `AgentPingPong` class.

```
@Override
public boolean start() {
    msgShard.signalAgentEvent(new AgentEvent(AgentEventType.AGENT_START));
}
```

Figure 12: The essential code for the signaling start event stage

Sending messages The `send()` method of the agent now simply delegates to the messaging shard. Internally, the shard adds the agent's address to the wave if it is missing, and then forwards the message through either a `wavePylon` or a `classicPylon`, depending on what it has.

Figure 13 presents the essential code for the sending messages stage. The complete implementation can be seen in the `AgentPingPong` class.

```
msgShard.sendMessage(wave);
```

Figure 13: The essential code for the sending messages stage

Receiving messages When a message arrives for the agent, the shard uses the proxy provided during configuration. The proxy forwards the wave to the agent's `processEvent()` method, which it was discussed in the previous example.

Figure 14 presents the essential code for the receiving messages stage. The complete implementation can be seen in the `AgentPingPong` class.

```
public boolean postAgentEvent(AgentEvent event) {  
    return processEvent(event)  
}
```

Figure 14: The essential code for the receiving messages stage

Figure 15 presents a sequence diagram containing the agents, the pylon, and the shards.

Figure 16 presents an UML class diagram for the agent - shard - pylon - pylon proxy - shard container structuring, especially from a context perspective.

2.3.4 Starting the deployment

To start the deployment, one can run the `Boot` class from the `simplePingPong` subpackage under `src-examples/example`. In this file, one can select the desired interaction with the communication infrastructure, either directly using `AgentPingPongPlain`, or via a `messagingShard` by means of `AgentPingPong`.

Regardless of the option, the node, the names of the agents involved, the configuration of the `sendTo` parameter with the names of the agents that will receive messages, as well as the number of pings sent must be provided.

3 Maven

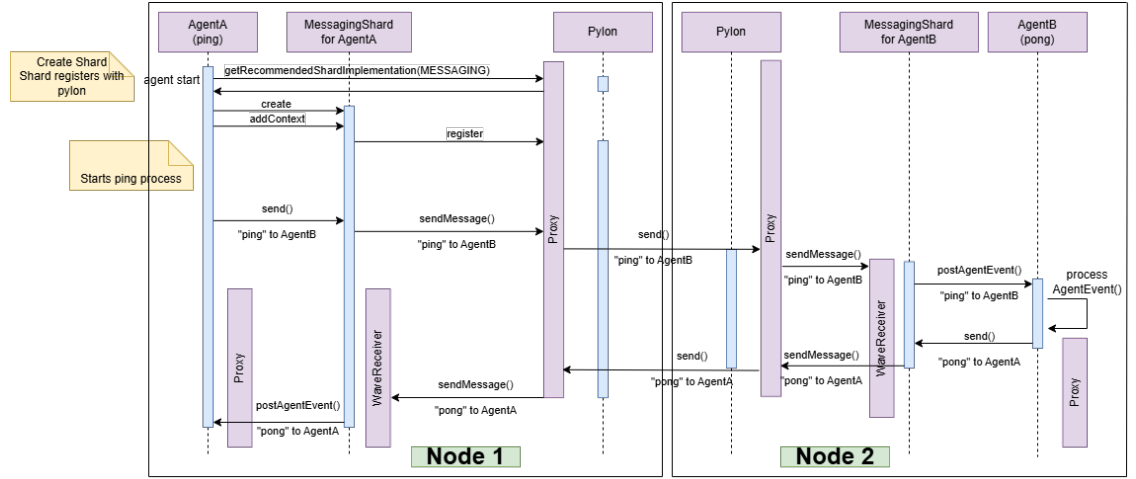


Figure 15: Sequence diagram containing the agents, the pylon, and the shards.

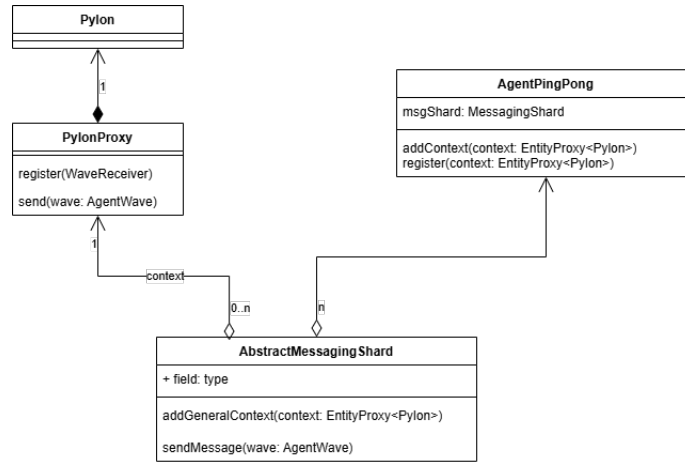


Figure 16: UML class diagram for the agent - shard - pylon - pylon proxy - shard container structuring, especially from a context perspective.

3.1 Build

To build the FLASH-MAS project via Maven, one can run in the root directory of the project, the commands below.

To build the project using the integrated profile `build-only`, run:

```
mvn clean install -P build-only
```

To build the project using plain Maven, run:

```
mvn clean install -DskipTests
```

3.2 Tests

Figure 17 presents a maven test command that should be run in the root directory of FLASH-MAS .

```
mvn test -P tests-only
```

Figure 17: Maven command for running tests

3.3 Build and tests

Figure 18 presents a maven command for build and tests that should be run in the root directory of FLASH-MAS .

```
mvn clean install
```

Figure 18: Maven command for build and tests

3.4 Configuration of source directories

Source directories can be configured in `pom.xml` file, in the `build-helper-maven-plugin` by adding a new element as in figure 19.

```
<configuration>
  <sources>
    <source>src</source>
    <source>src-examples</source>
    <source>src-tests</source>
    <source>src-experiments</source>
  </sources>
</configuration>
```

Figure 19: Maven commands for build

3.5 Upgrade of dependencies

The upgrade of dependencies is done in 2 ways, depending on how they were configured in `pom.xml`.

There are dependencies that have a BOM available, which means that in the `dependencyManagement` section in the `pom.xml` file, the respective BOM will be updated to the desired version, and thus all the covered dependencies will be upgraded. Such dependencies are Dependencies with BOM are configured in the `pom.xml` file in

For the other dependencies that do not have a BOM available, the upgrade is done specifically for each one.

3.6 Generating a jar with all dependencies

To generate a `.jar` with all dependencies, the `maven-assembly-plugin` is necessary, being present in `pom.xml` file.

Figure 20 presents the maven command used to generate the `.jar` with all dependencies that should be run in the root directory of FLASH-MAS .

```
mvn clean package
```

Figure 20: Maven command used to generate the jar with all dependencies